

Runir Module-Program Syntax and Semantics (Indexical Concept Policies)

Dominik Drexler
dominik.drexler@liu.se

1 Used Symbols

Symbol	Meaning
$\langle X \rangle$	Runir syntax placeholder for the serialization of semantic entity X ; monospaced text is literal syntax
P, Q	one grounded planning task and a generalized-planning class of tasks
$M_P = \langle S_P, A_P, T_P, s_0, S_G \rangle$	finite deterministic state model induced by task P
S_P, S_G	reachable planning states and the subset of goal states
A_P, T_P	ground actions and transition relation $T_P \subseteq S_P \times A_P \times S_P$
s, s', s_0, a	concrete states, initial state, and ground action, as determined by context
$s \xrightarrow{a} s'$	concrete transition from s to s' under action a
$\Pi = \langle m_0, \mathcal{M} \rangle$	module program with $\mathcal{M} = \{M\}$ and entry symbol $m_0 = m$
$M = \langle m, A, R, q_0, Q, \Phi, \mathcal{R} \rangle$	sole module with symbol, reserved empty list $A = \emptyset$, registers, entry memory, memories, features, and rules
m, q, q', q_0	module symbol and memory-state symbols, as determined by context
$A, R, Q, \Phi, \mathcal{R}$	reserved empty list, category-local ordered register lists, memory-state set, local-feature set, and rule set
$\mathbb{B}, \mathbb{N}$	boolean truth values and natural numbers; numerical values may additionally be ∞
B, N	boolean and numerical feature expressions
f, f_B, f_N, f_C, f_R	local feature symbols, optionally distinguished by boolean, numerical, concept, or role category
r, o	concept and role register symbols; r instead denotes a rule symbol when used as a rule
ρ, ρ_C, ρ_R	register valuation and its concept and role components
$\xi = (m, q, \rho, s)$	execution configuration with module, memory, register valuation, and planning state
$C, E, C_e, R_e, C_\times, E_\times$	condition list, effect list, concept expression, role expression, extended concept, and extended effect
$x_i, d, (d_1, d_2), \mathcal{I}$	action argument feature symbol, selected object or pair, and relational interpretation used by feature expressions
i	finite positional index
n_r, n_q, n_F, n_R	finite numbers of registers, memory states, feature declarations, and rules, respectively
n_{body}, n_x	finite numbers of alternative rule bodies and action arguments, respectively

Top goal: Generality—learn a module program that proves on the scaled training frontier, executes beyond it, and terminates structurally rather than merely fitting current tasks; **Soft goal 1: Total syntactic feature complexity**—among equally general correct programs, minimize the sum of the DL constructor-node counts of all declared feature expressions; **Soft goal 2: Memory-state control**—with equal secondary priority, maximize control flow through memory states so Boolean/numerical SCC projections stay small and the complete structural termination proof becomes maximally efficient.

2 State Models

A planning task P induces a finite deterministic state model

$$M_P = \langle S_P, A_P, T_P, s_0, S_G \rangle,$$

where S_P is the set of reachable states, A_P is the set of ground actions, $T_P \subseteq S_P \times A_P \times S_P$ is the transition relation, $s_0 \in S_P$ is the initial state, and $S_G \subseteq S_P$ is the set of goal states. We write $s \xrightarrow{a} s'$ for $(s, a, s') \in T_P$.

A generalized planning problem is a class $\mathcal{Q}$ of tasks with a common vocabulary and action schemas. A Runir module program denotes one finite-state controller that must solve every solvable task in $\mathcal{Q}$.

3 Module Programs

A module program is a tuple

$$\Pi = \langle m_0, \mathcal{M} \rangle,$$

with exactly one module: $\mathcal{M} = \{M\}$ and $m_0 = m$, where m is the symbol of M .

A module is a tuple

$$M = \langle m, A, R, q_0, Q, \Phi, \mathcal{R} \rangle,$$

where m is its symbol, $A = \emptyset$ is a reserved empty list, R comprises ordered concept- and role-register lists, $q_0 \in Q$ is the entry memory state, Q is a finite set of memory states, Φ is a finite set of local features, and $\mathcal{R}$ is a finite set of rules. Register, memory, feature, and rule symbols are local to the module.

Form	Runir Syntax Schema	Meaning
Program	<code>(:program (:entry $\langle m \rangle$) $\langle M \rangle$)</code>	Declares the sole module as the entry module.
Module	<code>(:module (:symbol $\langle m \rangle$) (:arguments) (:registers $\langle r_1 \rangle \dots \langle r_{n_r} \rangle$) (:entry $\langle q \rangle$) (:memory $\langle q_1 \rangle \dots \langle q_{n_q} \rangle$) (:features $\langle F_1 \rangle \dots \langle F_{n_F} \rangle$) (:rules $\langle R_1 \rangle \dots \langle R_{n_R} \rangle$) (:reset-spo $\langle P_1 \rangle \dots \langle P_{n_{spo}} \rangle$))</code>	Declares the sole finite-state controller.
Concept register	<code>(:concept $\langle r \rangle$)</code>	Declares a local concept register assignable by a matching load body.
Role register	<code>(:role $\langle o \rangle$)</code>	Declares a local role register assignable by a matching load body.
Memory	<code>(:memory $\langle q_1 \rangle \dots \langle q_{n_q} \rangle$)</code>	Declares the finite control states of the module.
Boolean feature	<code>(:boolean (:symbol $\langle b \rangle$) (:expression $\langle B \rangle$))</code>	Declares boolean feature symbol b with expression B .
Numerical feature	<code>(:numerical (:symbol $\langle n \rangle$) (:expression $\langle N \rangle$))</code>	Declares numerical feature symbol n with expression N .
Concept feature	<code>(:concept (:symbol $\langle c \rangle$) (:expression $\langle C \rangle$))</code>	Declares concept feature symbol c with expression C .
Role feature	<code>(:role (:symbol $\langle f_R \rangle$) (:expression $\langle R_e \rangle$))</code>	Declares role feature symbol f_R with expression R_e .
Rule	<code>(:rule (:symbol $\langle r \rangle$) (:expression (:source-memory $\langle q \rangle$) (:target-memory $\langle q' \rangle$) $\langle body_1 \rangle \dots \langle body_{n_{body}} \rangle$))</code>	Declares a control transition with one or more alternative bodies.
Precedence	<code>(:prec $\langle f_C \rangle \langle f_{C'} \rangle$)</code>	Declare the reset precedence relation $\langle f_C \rangle \prec \langle f_{C'} \rangle$ between concepts.

All module sections are mandatory and occur exactly in the displayed order; `(:arguments)` is always empty. Only the fields shown above are semantic fields. The single-module restriction makes only this module-level section empty. Registers, features, and rules may be empty; memory is nonempty. A program

contains exactly one module, and every `:rule` expression contains at least one body. Feature symbols are globally unique across feature categories within a module, and rule symbols are unique. Register symbols are unique within each category-local namespace; memory symbols are unique within the module. Identifiers match `[A-Za-z_][A-Za-z0-9_-]*`. Line comments start with `;` outside quoted strings and continue to the end of the physical line.

Each body inside one `:rule` is an alternative transition with the same source and target memories.

Each module may declare at most four concept registers and at most four role registers; the two limits are independent.

4 Feature Valuations

For each task P , module M , concrete state s , and register valuation ρ , each feature $f \in \Phi$ has a value $f_P(s, \rho)$. Concept, role, boolean, and numerical expressions are defined in `feature-syntax-antics.tex`. Inline feature expressions may occur only in feature declarations. Loads, conditions, effects, and action arguments refer to declared feature symbols.

Boolean features have values in $\mathbb{B}$. Numerical features have values in $\mathbb{N} \cup \{\infty\}$. Concept and role features denote complete sets of objects and object pairs.

5 Conditions

A condition is evaluated in the current valuation at s . Every condition references a declared local feature symbol. An empty `(:conditions)` section is true.

Condition	Runir Syntax Schema	Semantics
Positive boolean	<code>(positive)</code>	$b_P(s, \rho) = \text{true}$.
Negative boolean	<code>(negative)</code>	$b_P(s, \rho) = \text{false}$.
Greater than zero	<code>(greater_zero <n>)</code>	$n_P(s, \rho) > 0$.
Equal to zero	<code>(equal_zero <n>)</code>	$n_P(s, \rho) = 0$.

6 Extended Conditions and Effects

Extended conditions check whether an action argument belongs or doesn't belong to a concept, or whether the object stored in a register belongs or doesn't belong to a concept. Extended effects check whether an action argument or the object stored at a register enters or exits a concept. An object enters (resp. exits) a concept across a transition (s, s') if the object doesn't (resp. does) belong to the denotation of the concept at s , and it does (resp. doesn't) belongs to the denotation of the concept at s' .

Extended Condition	Runir Syntax Schema	Semantics
Belongs	<code>(belongs (:argument <x_i>) (:concept <f_C>))</code>	$x_i \in (f_C)_P(s, \rho)$.
Register belongs	<code>(belongs (:register (:concept <r>)) (:concept <f_C>))</code>	$\rho_C(r) \in (f_C)_P(s, \rho)$.
Not belongs	<code>(not-belongs (:argument <x_i>) (:concept <f_C>))</code>	$x_i \notin (f_C)_P(s, \rho)$.
Not register belongs	<code>(not-belongs (:register (:concept <r>)) (:concept <f_C>))</code>	$\rho_C(r) \notin (f_C)_P(s, \rho)$.

Extended Effect	Runir Syntax Schema	Semantics on s, s'
Enter	<code>(enter (:argument $\langle x_i \rangle$) (:concept $\langle f_C \rangle$))</code>	$\langle x_i \rangle \notin (f_C)_P(s, \rho) \wedge \langle x_i \rangle \in (f_C)_P(s', \rho)$.
Register enter	<code>(enter (:register (:concept $\langle r \rangle$)) (:concept $\langle f_C \rangle$))</code>	$\rho_C(r) \notin (f_C)_P(s, \rho) \wedge \rho_C(r) \in (f_C)_P(s', \rho)$.
Exit	<code>(exit (:argument $\langle x_i \rangle$) (:concept $\langle f_C \rangle$))</code>	$\langle x_i \rangle \in (f_C)_P(s, \rho) \wedge \langle x_i \rangle \notin (f_C)_P(s', \rho)$.
Register exit	<code>(exit (:register (:concept $\langle r \rangle$)) (:concept $\langle f_C \rangle$))</code>	$\rho_C(r) \in (f_C)_P(s, \rho) \wedge \rho_C(r) \notin (f_C)_P(s', \rho)$.

7 Rule Bodies

Let r be a rule with source memory q and target memory q' . A rule body is applicable in configuration ξ only if all of its conditions and extended conditions hold, and no object re-enters the same concept twice (checked using the concept histories in the configuration ξ). Bodies inside one rule are alternatives.

Body	Runir Syntax Schema	Semantics
Concept	<code>(:crule (:action "$\langle a \rangle$") (:arguments $\langle x_1 \rangle \dots \langle x_{n_x} \rangle$) (:conditions $\langle C \rangle$) (:xconditions $\langle C_\times \rangle$) (:xeffects $\langle E_\times \rangle$)))</code>	Branches over every transition $s \xrightarrow{a(o_1, \dots, o_{n_x})} s'$ such that s satisfies the conditions $\langle C \rangle$, the objects and registers satisfy the extended conditions $\langle C_\times \rangle$ in s , no object o_i re-enters the same concept, and the objects and registers satisfy the extended effects $\langle E_\times \rangle$ across (s, s') . Registers are unchanged. At least one argument must enter some concept.
Load concept	<code>(:load (:conditions $\langle C \rangle$) (:concept $\langle f_C \rangle$) (:register (:concept $\langle r \rangle$)))</code>	For every $d \in (f_C)_P(s, \rho)$, sets $\rho_C(r) = d$ without changing the concrete state.
Load role	<code>(:load (:conditions $\langle C \rangle$) (:role $\langle f_R \rangle$) (:register (:role $\langle o \rangle$)))</code>	For every $(d_1, d_2) \in (f_R)_P(s, \rho)$, sets $\rho_R(o) = (d_1, d_2)$ without changing the concrete state.

8 Execution

8.1 Configurations

An execution configuration is

$$\xi = (m, q, \rho, h, s),$$

where m is the sole module, q is the active memory state, ρ maps loaded registers to typed values, h is a history of for each concept, and $s \in S_P$ is the concrete planning state. The history of a concept is a subset of objects. Two configurations are distinct if any component differs. Initially execution starts in $(m, q_0, \emptyset, h_0, s_0)$, where q_0 is the module's entry memory, and h_0 is the empty history for each concept.

A rule is enabled only when its source memory is current. Load and Concept bodies enter its target memory and determine the remaining state and register effects.

8.2 Operational Semantics

A goal state terminates immediately. Otherwise, execution takes the union of the successors produced by every applicable Load and Concept body of every enabled rule. No body kind or rule suppresses successors produced by another. If this full union is empty, execution restores the caller when one exists; without

a caller, this is top-level failure at the non-goal state.

Within this union, universal `find_solution` branches over every applicable body, load value, matching ground action, module call, and immediate concept-rule successor. Existential `find_solution` samples one representative successor at each choice, so seeds may produce different rollouts. Universal traversal is the correctness semantics over all choices; existential traversal only obtains one successful witness or one counterexample and cannot establish correctness. Both modes consider only immediate successors; neither searches for a multi-step endpoint satisfying a body. An enabled Load whose referenced feature is empty is a failing branch. Caller restoration or the no-applicable-action fallback occurs only when the full successor union is empty.

A load body must target a register declared in the enclosing module and of the same type as the referenced feature. The selected value becomes available through `(c_register r)` or `(r_register o)` in later expressions of the same module execution.

The history of concepts are augmented with the objects that enter the concept across the taken transition. When a concept changes, the history of every concept that precedes it is reset (i.e., set to the empty set). Transitions across which an object re-enters the same concept are disabled. The relation defined by the precedence pairs in the reset specification must be a strict partial order; in particular, acyclic.

9 Termination

Reaching a planning goal preempts every body and successfully terminates the whole program. Away from a goal, an empty full successor union restores the caller when one exists and is top-level failure otherwise. A maximal program trajectory is a finite or infinite sequence of configurations generated by enabled rule bodies, immediate concrete transitions, loads, calls, and caller restorations.

Runir proves structural termination of the sole module on a finite abstract policy graph whose vertices combine memory states with qualitative boolean and numerical feature valuations. Within each strongly connected component, the Sieve algorithm repeatedly removes an edge justified by an unopposed strict numerical increase or decrease, or an unopposed boolean change. A module is structurally terminating when no cycle survives.

The public analysis defaults are `max_features=10` and `use_incomplete_preprocessing=True`. Incomplete preprocessing cheaply removes rules already justified by local qualitative changes before the full Sieve check. Exceeding `max_features` in a residual memory component is an analysis error, not a nontermination witness.

Rule effects determine the abstract changes used by Sieve. A Load preserves features that do not reference its target register and leaves register-dependent features unconstrained.

A module program passes the structural check iff its sole module passes. A reported counterexample is a surviving cycle in the abstract termination graph, not a concrete execution trace; it identifies the memory states, rules, and feature changes that still require a ranking argument or structural redesign.

10 Correctness

A module program Π solves a task P iff every maximal trajectory from the initial configuration is finite and terminates successfully in a state $s \in S_G$. Therefore correctness is universal over all nondeterministic choices left by sketches, loads, actions with the same denoted action arguments, and applicable rules. A module program solves $\mathcal{Q}$ iff it solves every solvable $P \in \mathcal{Q}$.

Source

This document follows the generalized-planning qualitative feature-rule model of arXiv:2101.00692 and extends it with Runir module-program control states, registers, and concrete action bodies.